

Assignment #01

Pooool Game

Design and implementation report

Academic year 2025-2026 - Concurrent and Distributed Programming

This report documents the submitted implementation and its concurrency choices. The design follows the terminology and principles in merged.pdf and the assignment README.

1. Problem analysis

Poool combines a reactive user interface, autonomous players and a physics simulation containing potentially thousands of moving balls. The shared logical state is the board: positions, velocities, scores and termination status must describe one consistent simulation step. Rendering must never observe a half-applied collision, while keyboard and bot impulses may arrive asynchronously.

The implementation separates input, simulation and presentation. GameEngine is the only owner of mutable game state. Its public operations are synchronized, so a step and an impulse are atomic with respect to each other. The Swing event dispatch thread only consumes immutable GameSnapshot values.

2. Architecture

Component	Responsibility	Concurrency rule
GameEngine	Physics, holes, score and termination	Single state owner; synchronized boundary
PlatformThreadRunner	Persistent default/platform worker threads	Private monitor with job queue and completion condition
ExecutorRunner	Task-based integration	Fixed ExecutorService and independent ranges
PooolApplication	Lifecycle, 60 Hz steps and asynchronous bot	Swing timer plus one bot platform thread
PooolView	Keyboard input and rendering	Runs on Swing EDT; renders snapshots

The two required variants share game rules but differ exactly at task execution. The explicit-thread version uses a monitor implemented with synchronized, wait and notifyAll. The task-based version submits the same independent ranges to a fixed executor. A SequentialRunner is retained only as a measurement baseline.

3. Behavior and game rules

- Arrow keys add an instantaneous velocity impulse to the human ball; the bot produces independent random impulses.
- Each simulation step integrates movement and friction, reflects balls at boundaries, resolves elastic equal-mass collisions and then checks holes.
- A small ball scores only when its last relevant collision was with a player ball. A later small-ball collision clears attribution.
- Pocketing a player ball immediately gives victory to the other player. Removing all small balls selects the highest score, or a draw.

A compact Petri-net reading is: place Ready holds one step token; transition Integrate forks worker tokens; place Integrated enables ResolveCollisions; transition ResolveCollisions produces ConsistentBoard; transition Render consumes a snapshot token and returns Ready. InputImpulse is concurrent with rendering but is serialized at the GameEngine monitor. This makes the step boundary explicit and prevents collision resolution from racing with integration.

4. Performance evaluation

PerformanceCheck creates the same deterministic board for each implementation. The following reproducible local run used Java 21, 8 available processors, 500 small balls and 30 steps. Times are wall-clock values and should be repeated on the target machine after warm-up for a formal evaluation.

Implementation	Time	Speedup vs sequential
Sequential	92.54 ms	1.00x
Platform threads	78.51 ms	1.18x
Executor	77.39 ms	1.20x

The result shows a modest benefit, not linear scaling. Position integration is parallel, but collision resolution and state publication are intentionally serialized to keep the implementation understandable and deterministic. By Amdahl's law, this serial $O(n^2)$ portion limits exploitation of the eight cores. The persistent worker pool and Executor avoid creating threads per frame; the Executor is slightly faster in this run.

5. Verification

JUnit executes the same progress invariant for sequential, platform-thread and Executor engines: a step completes, preserves the expected active population and does not terminate a fresh game. The source also contains a small finite ScoreModel for JPF. Its synchronized pocket operation checks non-negative scores under interleavings; keeping the model finite avoids exploring the full GUI and continuous physics state.

Useful additional JPF properties are: exactly one score changes per attributed pocket, no score changes after a small-ball-to-small-ball attribution reset, and a pocketed player ball fixes the opposite winner. The production build is verified with mvn test.